

Reporte de Práctica

Comunicación I2C entre 4 Arduinos

ALUMNOS: IVAN LUNA , DANIEL BUSTAMANTE

FECHA:22/09/2026

INSTITUTO TECNOLOGICO DE MAZATLAN

Objetivo

Comprender el funcionamiento del bus I2C mediante la comunicación entre un Arduino maestro y tres Arduinos esclavos, que comparten las líneas SDA y SCL y se identifican con direcciones distintas (0x08, 0x09 y 0x0A). Se busca que el alumno sea capaz de armar el bus, asignar direcciones y programar el envío y la recepción de datos entre los dispositivos.

Descripción de la práctica

Se armó en Tinkercad un bus I2C con cuatro Arduino Uno R4 WiFi. El maestro se comunica con los esclavos a través de las líneas SDA (A4) y SCL (A5), con tierra (GND) común entre todos los dispositivos.

- Esclavo 1 (0x08): recibe una orden del maestro y enciende o apaga un LED.
- Esclavo 2 (0x09): recibe un ángulo del maestro y mueve un servomotor.
- Esclavo 3 (0x0A): lee un potenciómetro y envía su valor al maestro cuando este se lo pide.

Cada 500 ms, el maestro pide el valor del potenciómetro al esclavo 3, lo convierte a un ángulo de 0° a 180° y se lo envía al esclavo 2. Además, desde el Monitor serie se puede escribir 1 o 0 para encender o apagar el LED del esclavo 1. El maestro muestra en el Monitor serie el valor leído y avisa si algún esclavo no responde.

Material utilizado

Hardware (simulado en Tinkercad)

- 4 Arduino Uno R4 WiFi
- 1 protoboard (Breadboard Small)
- 1 LED
- 1 resistencia de 470 Ω (LED, ajustada para el R4)
- 1 micro servo

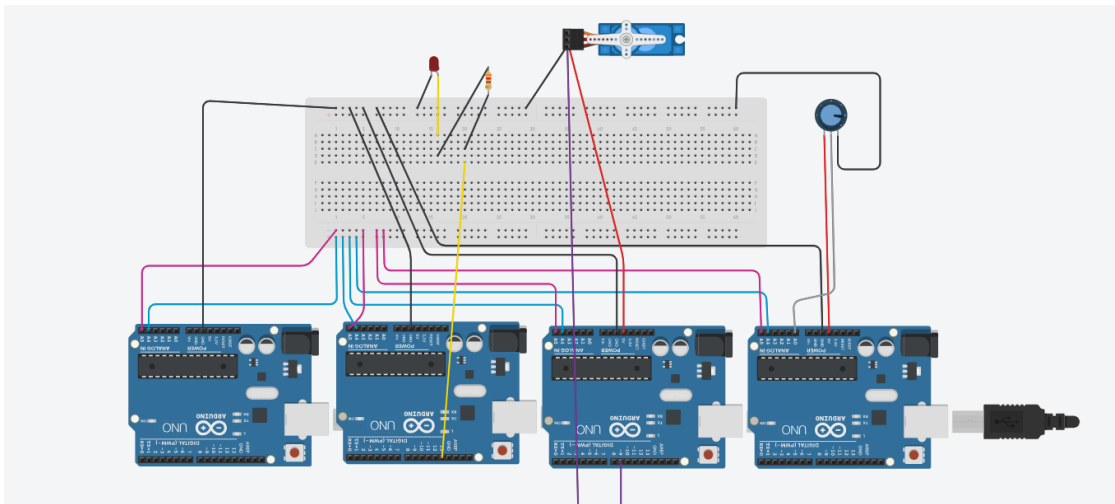
- 1 potenciómetro
- Cables de conexión
- 2 resistencias de 4.7 k Ω (pull-up en SDA y SCL, necesarias en el R4)

Software

- Librerías Wire y Servo (incluidas en el entorno de Arduino)
- Monitor serie

Diagrama de conexión

El diagrama muestra las conexiones utilizadas para implementar el bus I2C entre las cuatro placas, así como el LED, el servomotor y el potenciómetro.



Consideraciones específicas para el Arduino UNO R4 WiFi

El código de referencia fue escrito originalmente para Arduino UNO R3. Al migrar a la placa R4 WiFi se identificaron los siguientes cambios necesarios:

1. Monitor serie: en el R4 el puerto serie es USB nativo, y los primeros mensajes se pierden si el Monitor abre tarde. Se agregó al maestro una línea que espera a que se abra el puerto (`while (!Serial && millis() < 3000)`).
2. Resistencia del LED: en el R4 se utiliza 470 Ω en lugar de 220 Ω , ya que sus pines soportan menos corriente que los del R3 (del orden de 8 mA). Con 220 Ω , el LED demandaría cerca de 14 mA.
3. Pull-ups: en físico, se agrega una resistencia de 4.7 k Ω de SDA a 5V y otra de SCL a 5V. En el R4 ya no son opcionales, pues las placas no traen pull-ups fuertes.
4. Tierra y alimentación: se unieron los GND de las cuatro placas, pero no se conectaron los 5V entre ellas; cada placa se alimenta por su propio puerto USB.
5. Placas necesarias: se requieren 4 placas por equipo, por lo que lo recomendable es formar equipos de 4 integrantes.

Código

A continuación se presenta el código correspondiente a cada uno de los cuatro Arduinos, ya adaptado para el UNO R4 WiFi.

Maestro

```
#include <Wire.h>

const uint8_t DIR_LED    = 0x08; // Esclavo 1
const uint8_t DIR_SERVO  = 0x09; // Esclavo 2
const uint8_t DIR_POT    = 0x0A; // Esclavo 3

unsigned long ultimaLectura = 0;

void setup() {
  Serial.begin(9600);
  while (!Serial && millis() < 3000) { } // espera el USB del R4 (max. 3 s)
  Wire.begin(); // sin direccion = maestro
  Serial.println("Maestro listo. Escribe 1 (LED encendido) o 0 (LED apagado)");
}

void loop() {
  // --- Orden por el Monitor serie: enciende/apaga el LED del esclavo 1 ---
  if (Serial.available() > 0) {
    char c = Serial.read();
    if (c == '1' || c == '0') {
      byte orden = (c == '1') ? 1 : 0;
      Wire.beginTransmission(DIR_LED);
      Wire.write(orden);
      if (Wire.endTransmission() != 0) {
        Serial.println("Esclavo 1 (LED) no responde");
      }
    }
  }

  // --- Cada 500 ms: pide el potenciómetro al esclavo 3 y mueve el servo ---
  if (millis() - ultimaLectura >= 500) {
    ultimaLectura = millis();
    if (Wire.requestFrom(DIR_POT, 2) == 2) {
      byte alto = Wire.read();
      byte bajo = Wire.read();
      int valor = (alto << 8) | bajo; // 0 a 1023
      byte angulo = map(valor, 0, 1023, 0, 180);
      Wire.beginTransmission(DIR_SERVO);
      Wire.write(angulo);
      if (Wire.endTransmission() != 0) {
        Serial.println("Esclavo 2 (servo) no responde");
      }
      Serial.print("Potenciómetro: ");
      Serial.print(valor);
      Serial.print(" -> servo a ");
      Serial.print(angulo);
      Serial.println(" grados");
    } else {
      Serial.println("Esclavo 3 (potenciómetro) no responde");
    }
  }
}
```

Esclavo 1 — LED (dirección 0x08)

```
#include <Wire.h>

const uint8_t PIN_LED = 13;

void setup() {
  pinMode(PIN_LED, OUTPUT);
  Wire.begin(0x08); // direccion de este esclavo
  Wire.onReceive(recibir);
}

void loop() {
}

void recibir(int cuantos) {
  while (Wire.available()) {
    byte dato = Wire.read();
    digitalWrite(PIN_LED, dato ? HIGH : LOW);
  }
}
```

Esclavo 2 — Servomotor (dirección 0x09)

```
#include <Wire.h>
#include <Servo.h>

Servo miServo;
volatile byte angulo = 90;
volatile bool hayDato = false;

void setup() {
  miServo.attach(9);
  miServo.write(90);
  Wire.begin(0x09);
  Wire.onReceive(recibir);
}

void loop() {
  // El servo se mueve aqui, no dentro de la rutina de recepcion
  if (hayDato) {
    hayDato = false;
    miServo.write(angulo);
  }
}

void recibir(int cuantos) {
  while (Wire.available()) {
    int a = Wire.read(); // se lee UNA sola vez
    if (a > 180) a = 180;
    if (a < 0) a = 0;
    angulo = a;
    hayDato = true;
  }
}
```

Esclavo 3 — Potenciómetro (dirección 0x0A)

```
#include <Wire.h>

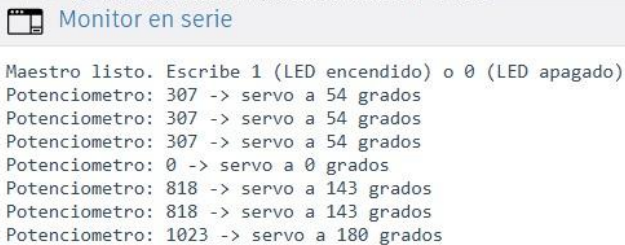
void setup() {
  Wire.begin(0x0A);
  Wire.onRequest(enviar);
}

void loop() {
}

void enviar() {
  int valor = analogRead(A0); // 0 a 1023
  Wire.write(highByte(valor)); // se manda en 2 bytes
  Wire.write(lowByte(valor));
}
```

Resultados

MONITOR SERIAL



```
Maestro listo. Escribe 1 (LED encendido) o 0 (LED apagado)
Potenciometro: 307 -> servo a 54 grados
Potenciometro: 307 -> servo a 54 grados
Potenciometro: 307 -> servo a 54 grados
Potenciometro: 0 -> servo a 0 grados
Potenciometro: 818 -> servo a 143 grados
Potenciometro: 818 -> servo a 143 grados
Potenciometro: 1023 -> servo a 180 grados
```

Durante las pruebas, la comunicación I2C funcionó correctamente entre el maestro y los tres esclavos, cada uno identificado con su dirección correspondiente. El maestro solicitó el valor del potenciómetro al esclavo 3 cada 500 ms, lo convirtió a un ángulo entre 0° y 180°, y lo envió de forma exitosa al esclavo 2, que movió el servomotor en consecuencia.

Asimismo, el envío de los caracteres '1' y '0' desde el Monitor serie permitió encender y apagar el LED controlado por el esclavo 1 sin inconvenientes. El maestro reportó correctamente en el Monitor serie tanto las lecturas del potenciómetro como los casos en los que algún esclavo no respondía, gracias a la verificación del valor devuelto por `endTransmission()` y `requestFrom()`.

Durante el desarrollo se detectaron y corrigieron dos errores comunes: una resistencia colocada con ambas terminales en la misma columna de la protoboard (lo que generaba un cortocircuito) y una lectura múltiple del mismo dato I2C dentro de una sola expresión, la cual devolvía valores inválidos a partir de la segunda lectura.

Conclusiones

6. El bus I2C permite comunicar varios dispositivos usando solo dos líneas (SDA y SCL) más tierra común. Agregar un esclavo no requiere pines adicionales, solo una dirección diferente.
7. Cada esclavo debe tener una dirección única. Si dos comparten la misma, ambos responden a la vez y los datos se corrompen.
8. El maestro controla la comunicación: decide con quién habla, cuándo y si pide o manda datos. El esclavo solo responde cuando se le llama.
9. La comprobación del resultado de `endTransmission()` permite detectar cuando un esclavo no contesta (NACK) en lugar de que el sistema se congele.
10. Se usó `millis()` en lugar de `delay()` en el maestro, de manera que atiende al Monitor serie y consulta al potenciómetro sin bloquearse.
11. Se identificaron dos errores frecuentes: una resistencia con ambas terminales en la misma columna de la protoboard, que provoca un cortocircuito, y la lectura repetida de un mismo dato I2C dentro de una expresión, que devuelve valores inválidos a partir de la segunda lectura.
12. Cada Arduino de la simulación necesita su propio programa, y los cuatro deben compartir la misma tierra (GND), sin unir sus líneas de 5V.
13. Migrar el código del Arduino UNO R3 al UNO R4 WiFi requirió ajustes en la resistencia del LED, la espera del puerto serie y la incorporación de resistencias pull-up físicas en las líneas SDA y SCL.